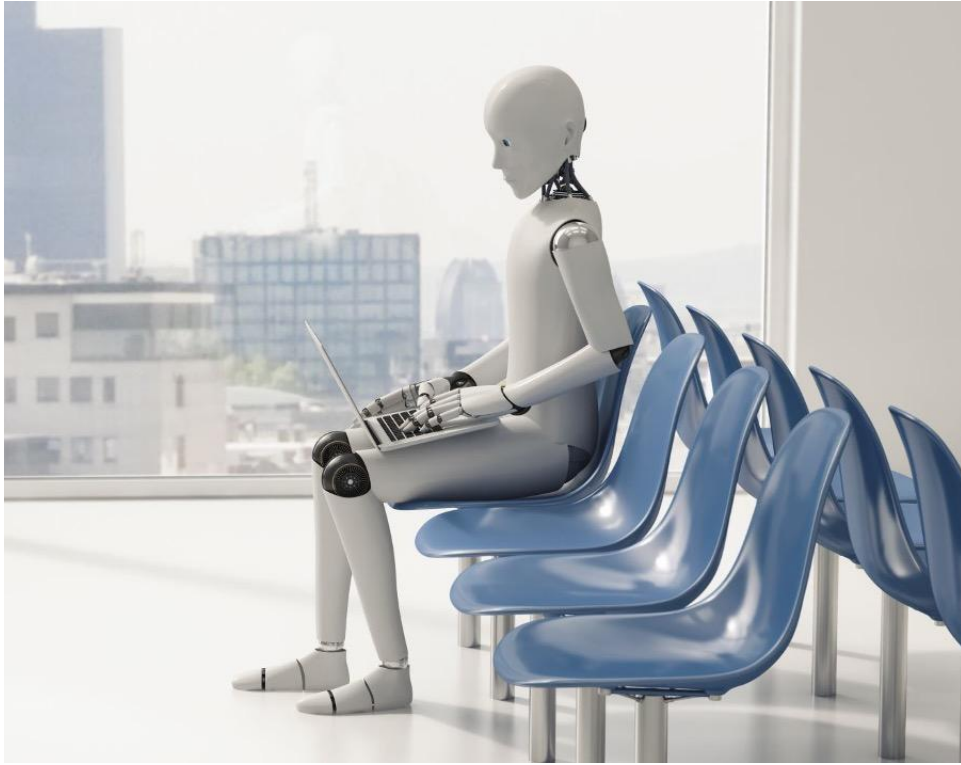




Capstone Project: End-to-End Banking Data Platform on Microsoft Fabric

Build an End-to-End Banking Data Platform on Microsoft Fabric — from Source Systems to a Governed Direct Lake Report

1. Capstone Overview

A retail bank is moving its reporting onto Microsoft Fabric. Its core banking system runs on a relational SQL database (customers, accounts, branches, loans, cards), and a Transaction API emits transaction events in near real time. The two sources are disconnected, and reporting today is slow and ungoverned.

In this capstone you will build one end-to-end solution: mirror the core banking SQL database into OneLake, ingest transaction events from the API into a Bronze layer, curate the data through a Bronze → Silver → Gold Medallion flow, serve a governed star-schema Gold model from a Fabric Warehouse with Row-Level and Column-Level Security, and finally publish an executive Power BI report in Direct Lake mode. The capstone brings together the topics you have already practised — Mirroring, API ingestion, Medallion architecture and Dataflow Gen2, the Lakehouse-versus-Warehouse decision, T-SQL stored procedures, cross-warehouse querying, RLS and CLS, and Direct Lake reporting.

2. Topics Integrated in this Capstone

Each phase exercises a specific skill from the program. Use this map to confirm you have covered every topic:

Topic / Module	Where it is demonstrated
Mirroring	Phase 1 – mirror the core banking SQL database into OneLake
API / pipeline ingestion	Phase 2 – ingest the transaction API into Bronze
Medallion architecture & Dataflow Gen2	Phase 3 – Bronze → Silver curation and profiling
Lakehouse vs Warehouse	Phase 3 – serving-layer decision
T-SQL stored procedures	Phase 4 – build and refresh the Gold star schema
Cross-warehouse / cross-item querying	Phase 4 – combine mirrored DB, Silver and Warehouse
Row-Level & Column-Level Security	Phase 4 – govern the Gold model
Direct Lake reporting	Phase 5 – semantic model and Power BI report

3. Source Systems Overview

The platform draws from two operational sources:

Source	Type	Description
Core Banking System	SQL Database (Mirrored)	Customers, accounts, branches, loans, cards
Transaction Service	REST / JSON API	Near-real-time transaction events

4. Sample Source Structure

A. Core Banking SQL Database — key tables

Column	Description
customer_id	Unique customer ID (PK)
full_name / email / phone	Customer name and contact details (email, phone are PII)
date_of_birth / gender / city	Customer demographics (date_of_birth is PII)
region	North, South, East, West, Central
customer_segment / kyc_status	Segment and KYC status
account_id / branch_id / account_type	Account attributes (Accounts table)
current_balance / account_status	Account balance and status (Accounts table)
branch_name / region / rm_login	Branch attributes (Branches table) – rm_login drives RLS
loan_id / loan_type / loan_amount	Loan attributes (Loans table)
outstanding_amount / loan_status	Loan exposure and status (Loans table)
card_id / card_number / card_type	Card attributes (Cards table) – card_number is PII

B. Transaction API — event payload (JSON)

Column	Description
transaction_id	Unique transaction / event ID
account_id	Linked account ID
event_timestamp	Event time (used for incremental ingestion)
transaction_type	Debit, Credit, UPI, NEFT, IMPS, ATM
amount / currency	Transaction amount and currency
status	Success, Failed, Pending
channel	Mobile App, Internet Banking, ATM, Branch
merchant_category	Retail, Travel, Utilities, Investment, Other

5. Capstone Phases

Phase 1: Mirror the Core Banking SQL Database

Objective

Bring the operational core banking SQL database into OneLake with near-real-time replication, without building manual ETL.

Activities

1. Configure Mirroring in Fabric for the core banking SQL database, selecting Customers, Accounts, Branches, Loans and Cards.
2. Verify that the mirrored tables appear in OneLake and are queryable through the mirrored database SQL endpoint.
3. Make a change in the source and confirm it is reflected in the mirror (incremental replication).

Expected Output

A mirrored database (e.g., `mirror_core_banking`) whose tables are continuously available in OneLake.

Phase 2: Ingest the Transaction API into Bronze

Objective

Land raw transaction events from the REST API into the Lakehouse Bronze layer, incrementally and with full lineage.

Activities

1. Create a Lakehouse (e.g., `bank_lakehouse`) to hold the Medallion layers.
2. Build a Data pipeline or Dataflow Gen2 that calls the transaction API, handling authentication and pagination.
3. Write the raw events into `bronze_transactions` with metadata columns `source_system`, `ingestion_timestamp` and `pipeline_run_id`, pulling incrementally by `event_timestamp`.

Expected Output

A `bronze_transactions` table populated from the API with raw structure and metadata preserved.

Phase 3: Build Silver in the Lakehouse

Objective

Profile the Bronze data and curate it into a clean, conformed Silver layer using Dataflow Gen2, then decide where the Gold layer is served.

Activities

1. Profile data quality across the mirrored tables and the API Bronze data (duplicates, missing values, invalid or negative amounts, inconsistent statuses and casing) and record a short profiling summary.
2. Use Dataflow Gen2 to build Silver tables — `silver_customers`, `silver_accounts`, `silver_branches`, `silver_loans`, `silver_transactions` — with deduplication, validation and standardization, conforming the API events to the mirrored account and customer keys.

Expected Output

Cleaned, conformed Silver tables and a one-paragraph serving-layer recommendation.

Phase 4: Build and Govern the Gold Model in the Warehouse

Objective

Build a star-schema Gold model in a Fabric Warehouse using T-SQL stored procedures, then secure it with Row-Level and Column-Level Security.

Activities

1. Create a Warehouse (e.g., `bank_warehouse`) with a `gold` schema, and build dimensions (`dim_customer`, `dim_account`, `dim_branch`, `dim_loan`, `dim_date`) and facts (`fact_transaction`, `fact_loan`).
2. Author parameterized T-SQL stored procedures to build and refresh the model, using cross-warehouse / cross-item queries (three-part naming) to combine the mirrored database and the Silver tables.
3. Apply Row-Level Security: create an RM-to-region mapping and a `SECURITY POLICY` with a `FILTER PREDICATE` so each RM sees only their region.
4. Apply Column-Level Security: `DENY SELECT` on PII columns — `card_number`, `email`, `phone`, `date_of_birth` — for the analyst role.
5. Validate the model and the security with two personas: a regional RM and a general analyst.

Expected Output

A populated Gold star schema built by stored procedures, with RLS and CLS enforced and verified.

Phase 5: Build the Direct Lake Report

Objective

Deliver an executive Power BI report over the Gold model using Direct Lake mode, with security flowing through to the report.

Activities

1. Build a semantic model in Direct Lake mode over the Gold tables and define the star-schema relationships.
2. Create DAX measures: total transaction value, transaction success rate, failed-transaction rate by channel, total balance and total outstanding.
3. Define security roles on the semantic model so RLS flows through to viewers.
4. Build a report answering the business questions in Section 6, and confirm the model runs in Direct Lake mode (no DirectQuery / Import fallback).

Expected Output

A published Direct Lake report on the governed Gold model, enforcing row-level security per viewer.

6. Report Requirements — Business Questions to Answer

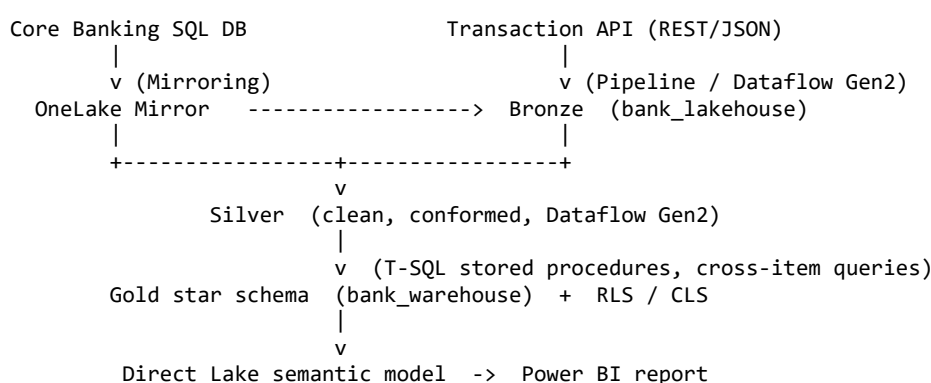
The Direct Lake report must enable business users to answer:

1. Which region and branch generate the highest transaction value and volume?
2. What is the success rate of digital channels, and which channel has the highest failure rate?
3. Which customer segments hold the highest balances and outstanding loan amounts?
4. Which loan types carry the highest outstanding exposure?
5. When a regional RM opens the report, do they see only their region (RLS through Direct Lake)?
6. Are PII fields (card number, email, date of birth) absent from the analyst report (CLS)?

7. Deliverables

Deliverable	Description
Mirrored Database	Replicated core banking tables in OneLake
Bronze API Ingestion	Pipeline / Dataflow and the populated bronze_transactions
Silver Layer + Profiling	Cleaned, conformed Silver tables and a data-quality summary
Gold Warehouse Model	Star schema with dimensions, facts and stored procedures
Security Implementation	RLS security policy and CLS grants, with validation evidence
Direct Lake Report	Semantic model, DAX measures and the published Power BI report
Architecture Diagram	Source-to-report data flow
Final Insights Summary	5–6 business and governance insights

8. Suggested Architecture Diagram



9. Evaluation Rubric

Criteria	Weightage
Mirroring of the core banking SQL database	15%
Transaction API ingestion into Bronze	15%
Medallion Silver curation and profiling (Dataflow Gen2)	20%
Gold star-schema model, T-SQL stored procedures and cross-warehouse query	20%
Row-Level and Column-Level Security	15%
Direct Lake semantic model and Power BI report	10%
Documentation and insights	5%

10. Bonus / Stretch Goals

1. Handle late-arriving / out-of-order API events with an incremental (CDC-style) load.
2. Add Dynamic Data Masking on PII columns in addition to Column-Level Security.
3. Add a data-quality scorecard page to the report.
4. Tune the Direct Lake semantic model (relationships, measures) and note the performance.

11. Final Submission Format

Submit one ZIP file containing:

```
capstone_banking_fabric/
|-- source/
|   |-- core_banking_schema.sql
|   `-- transaction_api_spec.md
|-- lakehouse/
|   |-- api_ingestion_pipeline.json
|   |-- bronze_to_silver_dataflow.pqt
|   `-- profiling_summary.xlsx
|-- warehouse/
|   |-- 01_dimensions.sql
|   |-- 02_facts.sql
|   |-- 03_stored_procedures.sql
|   |-- 04_cross_warehouse_query.sql
|   `-- 05_rls_cls.sql
|-- documentation/
|   |-- architecture_diagram.png
|   `-- final_insights_summary.docx
`-- README.md
```